

C std

Nathan Warner



Northern Illinois
University

Computer Science
Northern Illinois University
United States

Contents

1	Stdio	2
1.1	Output	2
1.2	Input	4
1.3	Binary I/O	6
1.4	Flushing	7
1.5	File Positioning	8
1.6	Eof and errors	9
1.7	Removing and renaming files	10
2	string.h	11
3	Memory operations in string.h	14
4	stddef.h	15
5	stdlib.h	16
5.1	Memory management	16
5.2	String to number	17
5.3	Program termination and process control	18
5.4	Random numbers	19
5.5	Sorting and searching	20
5.6	Integer Math helpers	21

Stdio

1.1 Output

-

```
0  int printf(const char *restrict format, ...);
```

Upon successful return, these functions return the number of bytes printed (excluding the null byte used to end output to strings).

On error, a negative value is returned, and `errno` is set to indicate the error.

-

```
0  int fprintf(FILE *stream, const char *format, ...);
```

Upon successful return, these functions return the number of bytes printed (excluding the null byte used to end output to strings).

On error, a negative value is returned, and `errno` is set to indicate the error.

-

```
0  int sprintf(char *str, const char *format, ...);
```

Returns the total number of characters successfully written, excluding the null terminator (`\0`).

If an error occurs, it returns a negative value and `errno` is set.

-

```
0  int snprintf ( char * s, size_t n, const char * format, ...  
    ↪ );
```

Returns the number of characters that would have been written if n had been sufficiently large, not counting the terminating null character.

If an encoding error occurs, a negative number is returned and `errno` is set.

-

```
0  int fputc(int c, FILE *stream);  
1  int putc(int c, FILE *stream);  
2  int putchar(int c);
```

Return the character written as an unsigned char cast to an int or EOF on error.

•

```
0  int fputs(const char *restrict s, FILE *restrict stream);  
1  int puts(const char *s);
```

Returns a nonnegative number on success, or EOF on error.

1.2 Input

-

```
0  int scanf(const char *restrict format, ...);
1  int fscanf(FILE *restrict stream, const char *restrict
   ↪ format, ...);
```

On success, these functions return the number of input items successfully matched and assigned; this can be fewer than provided for, or even zero, in the event of an early matching failure.

The value EOF is returned if the end of input is reached before either the first successful conversion or a matching failure occurs. EOF is also returned if a read error occurs, in which case the error indicator for the stream is set, and errno is set to indicate the error.

-

```
0  int getchar(void);
1  int fgetc(FILE *stream);
```

Upon successful completion, fgetc() shall return the next byte from the input stream pointed to by stream. If the end-of-file indicator for the stream is set, or if the stream is at end-of- file, the end-of-file indicator for the stream shall be set and fgetc() shall return EOF. If a read error occurs, the error indicator for the stream shall be set, fgetc() shall return EOF, and shall set errno to indicate the error.

The return behavior for getchar is the same as fgetc.

-

```
0  int sscanf(const char *restrict str, const char *restrict
   ↪ format, ...);
```

On success, these functions return the number of input items successfully matched and assigned; this can be fewer than provided for, or even zero, in the event of an early matching failure.

The value EOF is returned if the end of input is reached before either the first successful conversion or a matching failure occurs.

-

```
0  char *fgets(char *restrict s, int n, FILE *restrict stream);
```

Upon successful completion, `fgets()` shall return `s`. If the stream is at end-of-file, the end-of-file indicator for the stream shall be set and `fgets()` shall return a null pointer. If a read error occurs, the error indicator for the stream shall be set, `fgets()` shall return a null pointer, and shall set `errno` to indicate the error.

1.3 Binary I/O

-

```
0  size_t fwrite(const void *restrict data_ptr, size_t  
    ↪ element_size, size_t n_elements, FILE *restrict stream);
```

The `fwrite()` function shall return the number of elements successfully written, which may be less than `nitems` if a write error is encountered. If `size` or `nitems` is 0, `fwrite()` shall return 0 and the state of the stream remains unchanged. Otherwise, if a write error occurs, the error indicator for the stream shall be set, and `errno` shall be set to indicate the error.

-

```
0  size_t fread(void ptr[restrict size * n], size_t size,  
    ↪ size_t n, FILE *restrict stream);
```

The function `fread()` reads `n` items of data, each `size` bytes long, from the stream pointed to by `stream`, storing them at the location given by `ptr`.

On success, `fread()` and `fwrite()` return the number of items read or written. This number equals the number of bytes transferred only when `size` is 1. If an error occurs, or the end of the file is reached, the return value is a short item count (or zero).

1.4 Flushing

-

```
0  int fflush(FILE* stream);
```

Returns 0 on success. Returns EOF on failure.

1.5 File Positioning

-

```
o  int fseek(FILE* stream, long offset, int whence);
```

whence can be

- **SEEK_SET**: Beginning of file
- **SEEK_CUR**: Current position
- **SEEK_END**: End of file

Returns zero on success, nonzero on failure.

-

```
o  long ftell(FILE* stream);
```

returns the current file position. For binary files, this is commonly used as a byte offset.

Returns current position on success, -1L on failure.

-

```
o  void rewind(FILE* stream);
```

reset the file position indicator in a stream

1.6 Eof and errors

-

```
0  int feof(FILE *stream);
```

The feof() function shall return non-zero if and only if the end-of-file indicator is set for stream, otherwise returns zero.

-

```
0  int ferror(FILE* stream);
```

Returns nonzero if the error indicator is set, zero otherwise.

-

```
0  void clearerr(FILE* stream);
```

-

```
0  void perror(const char* s);
```

1.7 Removing and renaming files

-

```
0  int remove(const char* filename);
```

Returns zero on success, nonzero on failure.

-

```
0  int rename(const char* old, const char* new);
```

Returns 0 on success. Returns nonzero on failure.

string.h

- **Note:** string.h defines NULL and size_t described in stddef.h

-

```
0  size_t strlen(const char* s);
```

Returns the length of a C string, not counting the null terminator.

-

```
0  int strcmp(const char *s1, const char *s2);
```

A negative return value implies s1 is less than s2. A return value of zero implies s1 equals s2. A positive return value implies s1 is greater than s2.

-

```
0  int strncmp(const char* s1, const char* s2, size_t n);
```

Has same return behavior as strcmp.

-

```
0  char* strcpy(char* dest, const char* src);
```

Returns dest.

-

```
0  char *strncpy(char *dest, const char *src, size_t n);
```

Returns dest.

-

```
0  char *strcat(char *dest, const char *src);
```

Returns dest.

-

```
0  char *strncat(char *dest, const char *src, size_t n);
```

Returns dest

•

```
0 char* strchr(const char* s, int c);
```

Returns pointer to first occurrence of character *c* in string *s*, or NULL if not found.

•

```
0 char* strrchr(const char* s, int c);
```

Same return value as strchr

•

```
0 char* strstr(const char* haystack, const char* needle);
```

Returns pointer on success, NULL if not found.

•

```
0 char* strtok(char* str, const char* delim);
```

Splits a string into tokens using delimiter characters.

Returns pointer to next token (or first token).

Saves internal state (where it left off).

•

```
0 size_t strcspn(const char *s, const char *reject);
```

Returns the length of the initial segment of *s* containing no characters from reject.

•

```
0 char *strpbrk(const char *s, const char *accept);
```

Finds the first character in *s* that matches any character in accept. Or NULL if none is found.

-

```
0 char* strerror(int errnum);
```

Returns a human-readable error message for an error number.

Memory operations in string.h

-

```
o void* memset(void* s, int c, size_t n);
```

Sets n bytes of memory to the byte value c .

Returns s

-

```
o void *memcpy(void *dest, const void *src, size_t n);
```

Returns $dest$.

-

```
o void *memmove(void *dest, const void *src, size_t n);
```

Returns $dest$.

-

```
o int memcmp(const void *s1, const void *s2, size_t n);
```

Returns:

- < 0 first differing byte in $s1$ is less than the one in $s2$
- 0 all n bytes are equal
- > 0 first differing byte in $s1$ is greater than the one in $s2$

- o void* memchr(const void* s, int c, size_t n);

Returns pointer to the first matching byte or NULL.

stddef.h

- **Macros:**
 - NULL
- **Types:**
 - **ptrdiff_t**: Signed integer type of the result of subtracting two pointers.
 - **wchar_t**: Integer type whose range of values can represent distinct codes for all members of the largest extended character set specified among the supported locales; the null character shall have the code value zero. Each member of the basic character set shall have a code value equal to its value when used as the lone character in an integer character constant if an implementation does not define
 - **size_t**: Unsigned integer type of the result of the sizeof operator.

stdlib.h

5.1 Memory management

- **Macros:**

- **EXIT_FAILURE**: Unsuccessful termination for `exit()`; evaluates to a non-zero value.
- **EXIT_SUCCESS**: Successful termination for `exit()`; evaluates to 0.
- **RAND_MAX**: Maximum value returned by `rand()`; at least 32767.
- **MB_CUR_MAX**: Maximum number of bytes in a character specified by the current locale

- - `void* malloc(size_t size);`

Allocates `size` bytes of uninitialized memory.

Returns a pointer to the allocated memory, or NULL if allocation fails.

-

- - `void* calloc(size_t n, size_t size);`

Returns a pointer to the allocated memory, or NULL if allocation fails.

-

- - `void* realloc(void* ptr, size_t size);`

Returns a pointer to the resized block, or NULL if resizing fails.

- **free**

- - `void free(void* ptr);`

Returns nothing.

5.2 String to number

-

```
0  int atoi(const char* nptr);
1  long atol(const char* nptr);
2  long long atoll(const char* nptr);
```

Returns the converted integer.

-

```
0  long strtol(const char *restrict nptr, char **restrict
    ↪  endptr, int base);
1  long long strtoll(const char *restrict nptr, char **restrict
    ↪  endptr, int base);
2  unsigned long strtoul(const char *restrict nptr, char
    ↪  **restrict endptr, int base);
3  unsigned long strtoull(const char *restrict nptr, char
    ↪  **restrict endptr, int base);
4  double strtod(const char *restrict nptr, char **restrict
    ↪  endptr);
5  float strtof(const char *restrict nptr, char **restrict
    ↪  endptr);
6  long double strtold(const char *restrict nptr, char
    ↪  **restrict endptr);
```

Returns the converted value. If endptr is not NULL, it is set to the first character after the parsed number.

Errno is set on overflow / underflow.

5.3 Program termination and process control

-

```
o void exit(int status);
```

Returns nothing.

-

```
o void abort(void)
```

Returns nothing.

-

```
o int atexit(void (*func)(void));
```

Returns zero on success, nonzero on failure.

-

```
o void _Exit(int status);
```

-

```
o int system(const char* string);
```

Returns implementation-defined status information.

The return value is

- If command is NULL, then a nonzero value if a shell is available, or 0 if no shell is available.
- If a child process could not be created, or its status could not be retrieved, the return value is -1 and errno is set to indicate the error.
- If a shell could not be executed in the child process, then the return value is as though the child shell terminated by calling `_exit(2)` with the status 127.
- If all system calls succeed, then the return value is the termination status of the child shell used to execute command. (The termination status of a shell is the termination status of the last command it executes.)

```
• o char* getenv(const char* name);
```

Returns a pointer to the value string, or NULL if the variable does not exist.

5.4 Random numbers

-

```
0  int rand(void);
```

Returns a pseudo-random integer in the range 0 to $RAND_{MAX}$.

-

```
0  void srand(unsigned int seed);
```

seeds the pseudo-random number generator used by `rand`.

5.5 Sorting and searching

-

```
o void qsort(void *base, size_t nmemb, size_t size, int  
  ↪ (*compare)(const void *, const void *));
```

Sorts an array. Returns nothing

The comparison function must return:

- < 0 if first element comes before second
- 0 if elements compare equal
- > 0 if first element comes after second

```
• o void *bsearch(const void *key, const void *base, size_t  
  ↪ nmemb, size_t size, int (*compar)(const void *, const  
  ↪ void *));
```

Performs binary search on a sorted array.

Returns pointer to the matching element, or NULL.

5.6 Integer Math helpers

- ```
0 int abs(int j);
1 long labs(long j);
2 long long llabs(long long j);
3
4 div_t div(int numer, int denom);
5 ldiv_t ldiv(long numer, long denom);
6 lldiv_t lldiv(long long numer, long long denom);
```

- 

```
0 typedef struct {
1 int quot;
2 int rem;
3 } div_t;
```

Its variants are similar.